UAH Mars Drone ASW
Architecture Design Document

Index

5 Software Design

5.1 Software Static Architecture

5.2 Software Dynamic Architecture

5.2.1 Computational Model **5.2.2 Software Behaviour**

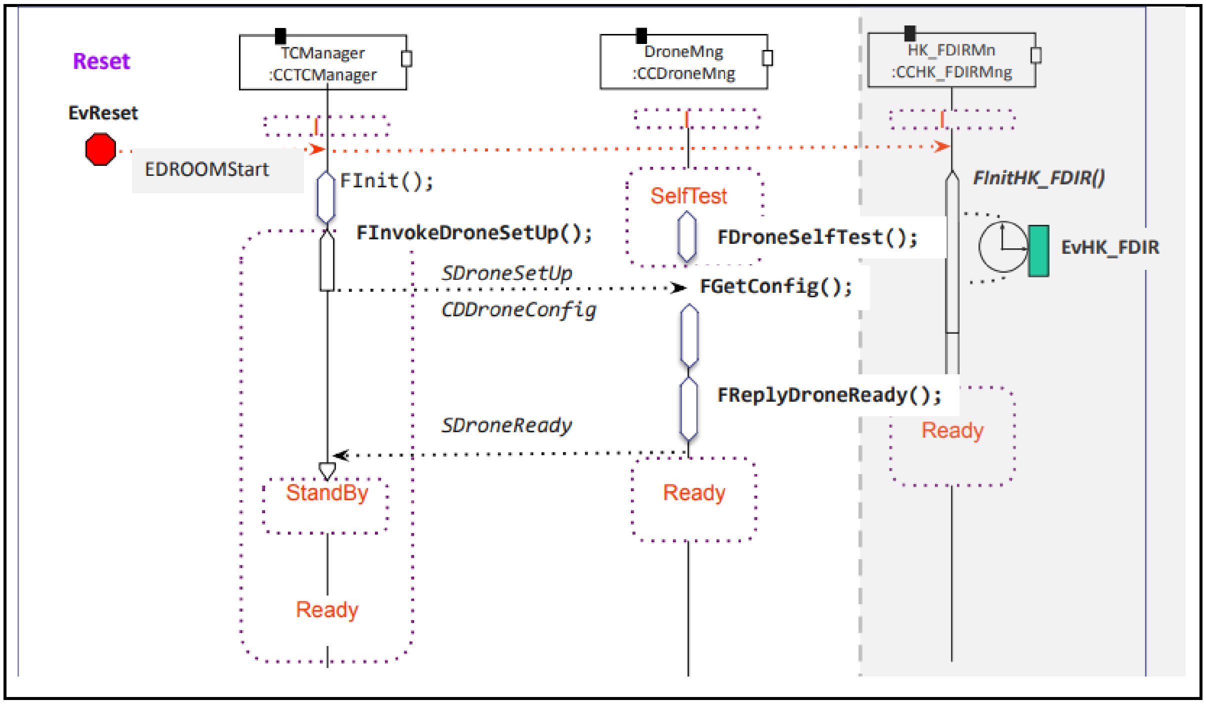
Event	Description
EvRxTC (external IRQ)	TC reception
EvAcceptedTC (internal)	TC acceptation
EvRejectedTC (internal)	TC rejection
EvToReboot (internal)	System Reboot
EvHK_FDIR (timing)	Periodic HK and FDIR Mng
EvHK_FDIR_TC (internal)	HK_FDIR TC Execution
EvReset (external)	System reset
EvBKGTC (internal)	Background TC execution

EvDroneTC (internal)	Drone TC execution
EvPrioTC (internal)	Priority TC execution
EvInitFlightPlan (internal)	Flight plan initialization
EvFlightCtrl (timing)	Flight control algorithm
EvFlightPlanDone (internal)	Flight plan done
EvDroneTCInFlight (internal)	In-flight drone TC execution

Scenarios

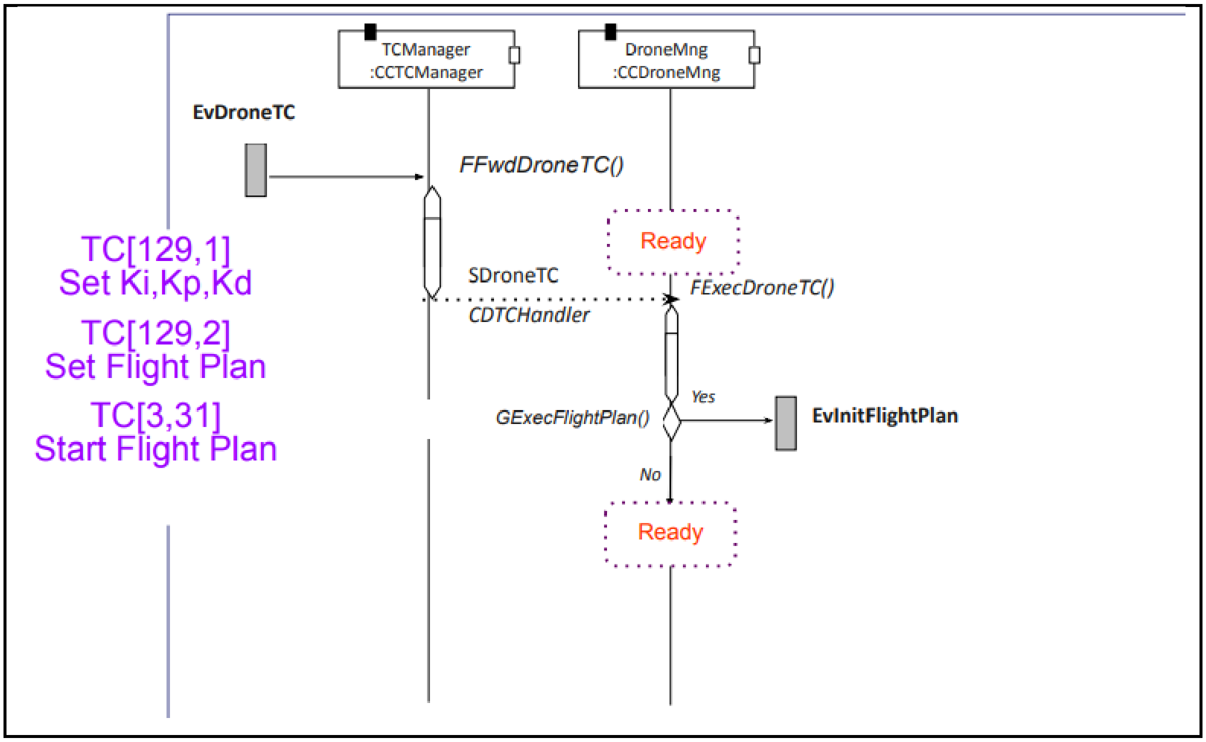
TODO 02 Añade los diagramas de secuencia de aquellos escenarios ***en los que participa el componente DroneMng***

EvReset



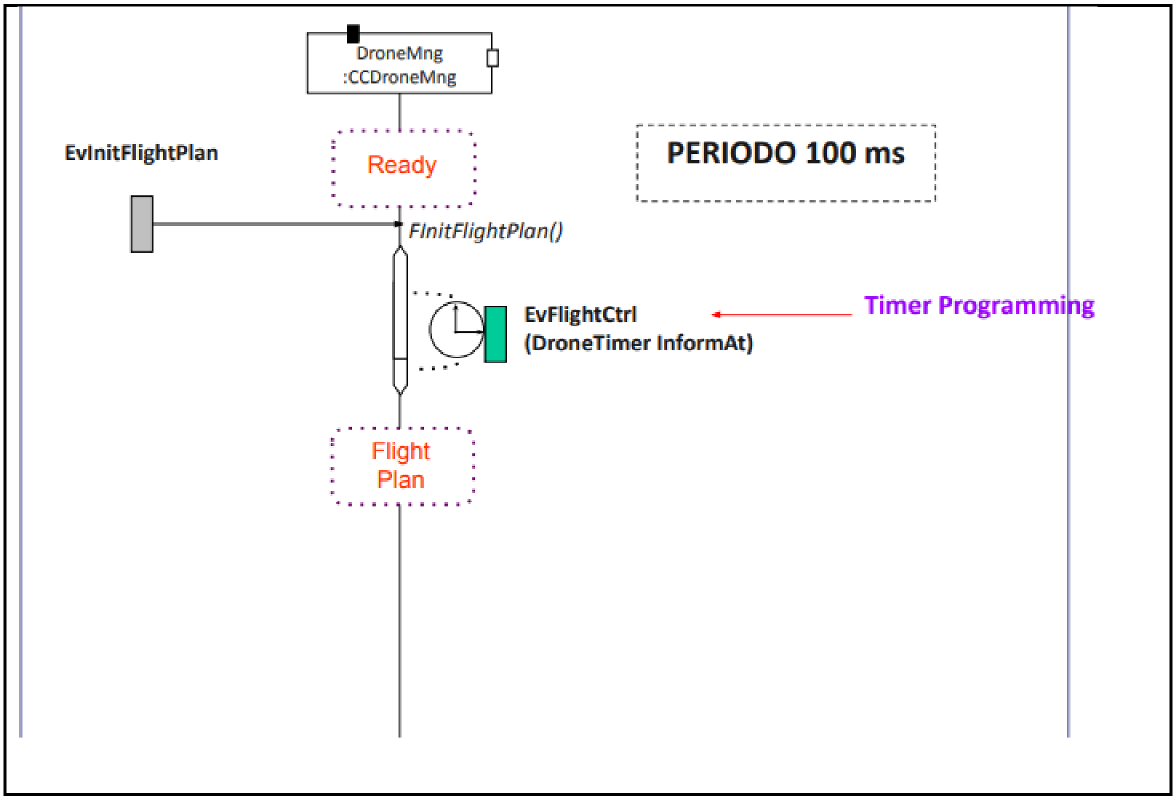
Reset
EvReset
EDROOMStart
TCManager
:CCTCManager
DroneMng
:CCDroneMng
HK_FDIRMn
:CCHK_FDIRMng
FInit();
FInvokeDroneSetUp();
SDroneSetUp
CDDroneConfig
SelfTest
FDroneSelfTest();
FGetConfig();
FInitHK_FDIR()
EvHK_FDIR
FReplyDroneReady();
SDroneReady
StandBy
Ready
Ready
Ready

EvDroneTC

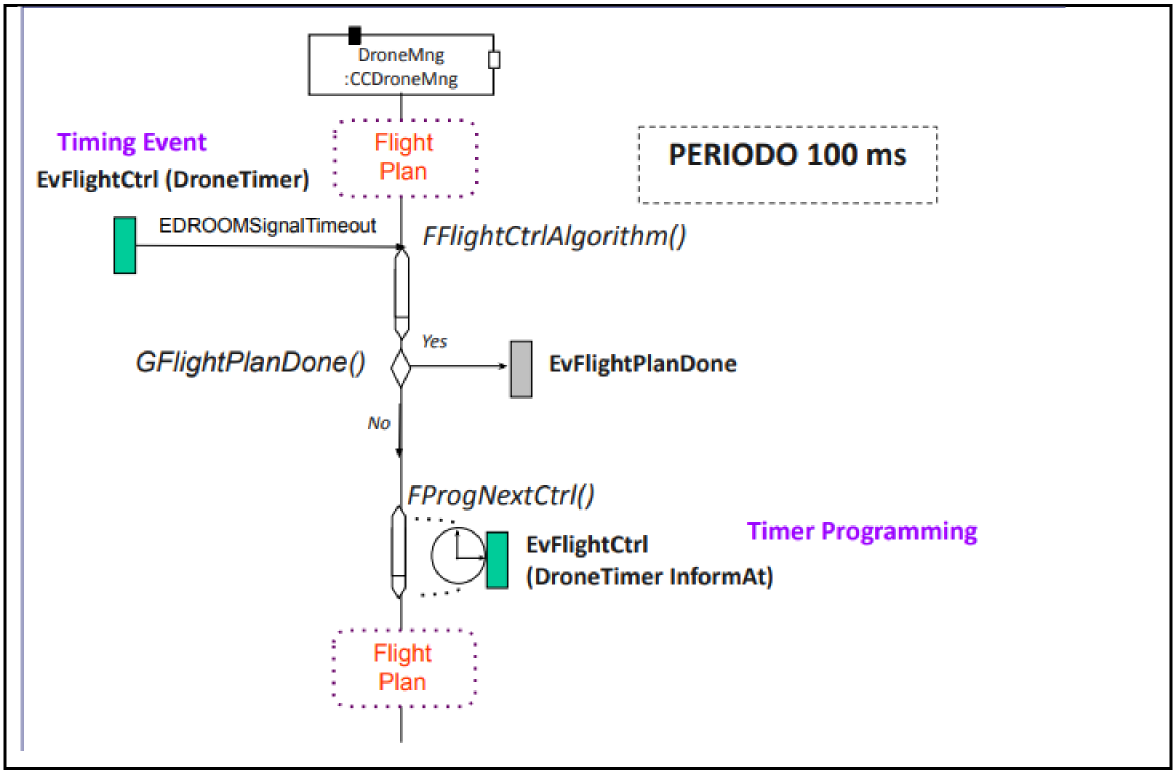


TCManager
:CCTCManager
DroneMng
:CCDroneMng
EvDroneTC
FFwdDroneTC()
TC[129,1]
Set Ki,Kp,Kd
TC[129,2]
Set Flight Plan
TC[3,31]
Start Flight Plan
Ready
SDroneTC
FExecDroneTC()
CDTCHandler
Yes
GExecFlightPlan()
EvInitFlightPlan
No
Ready

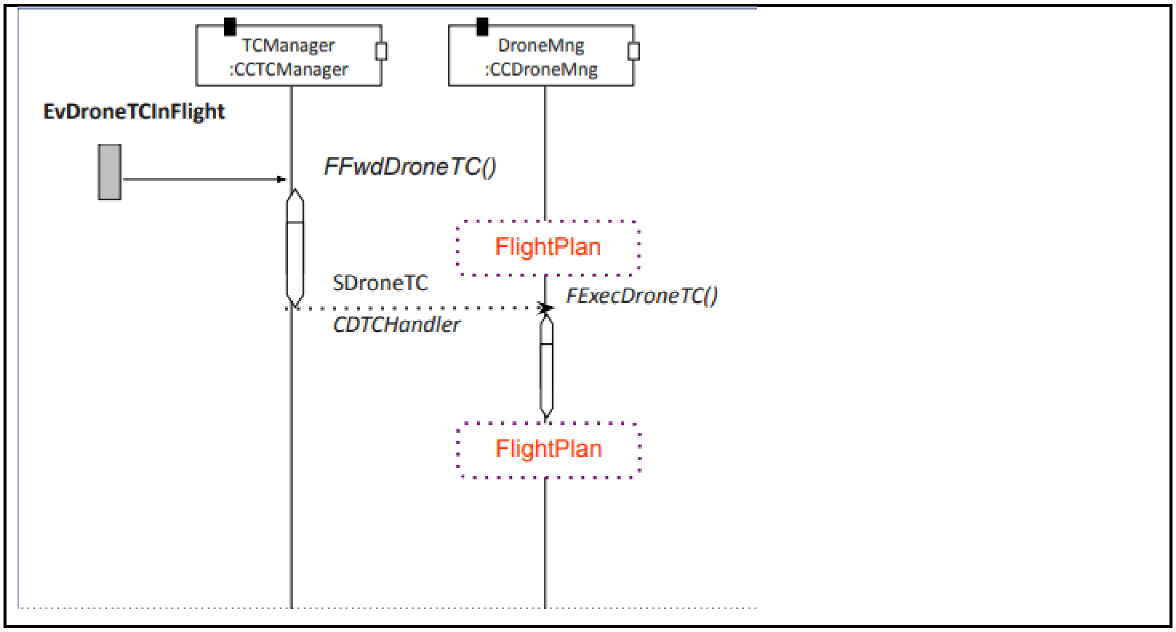
EvInitFlightPlan



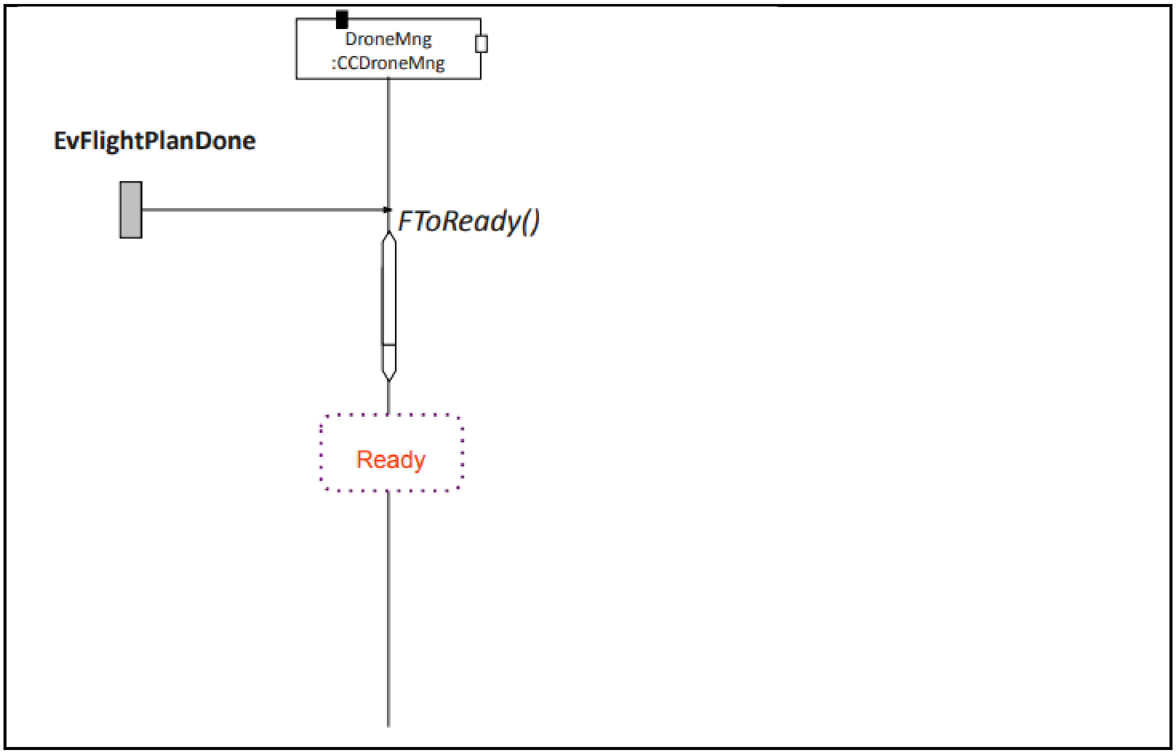
EvFlightCtrl



EvDroneTCInFlight



EvFlightPlanDone



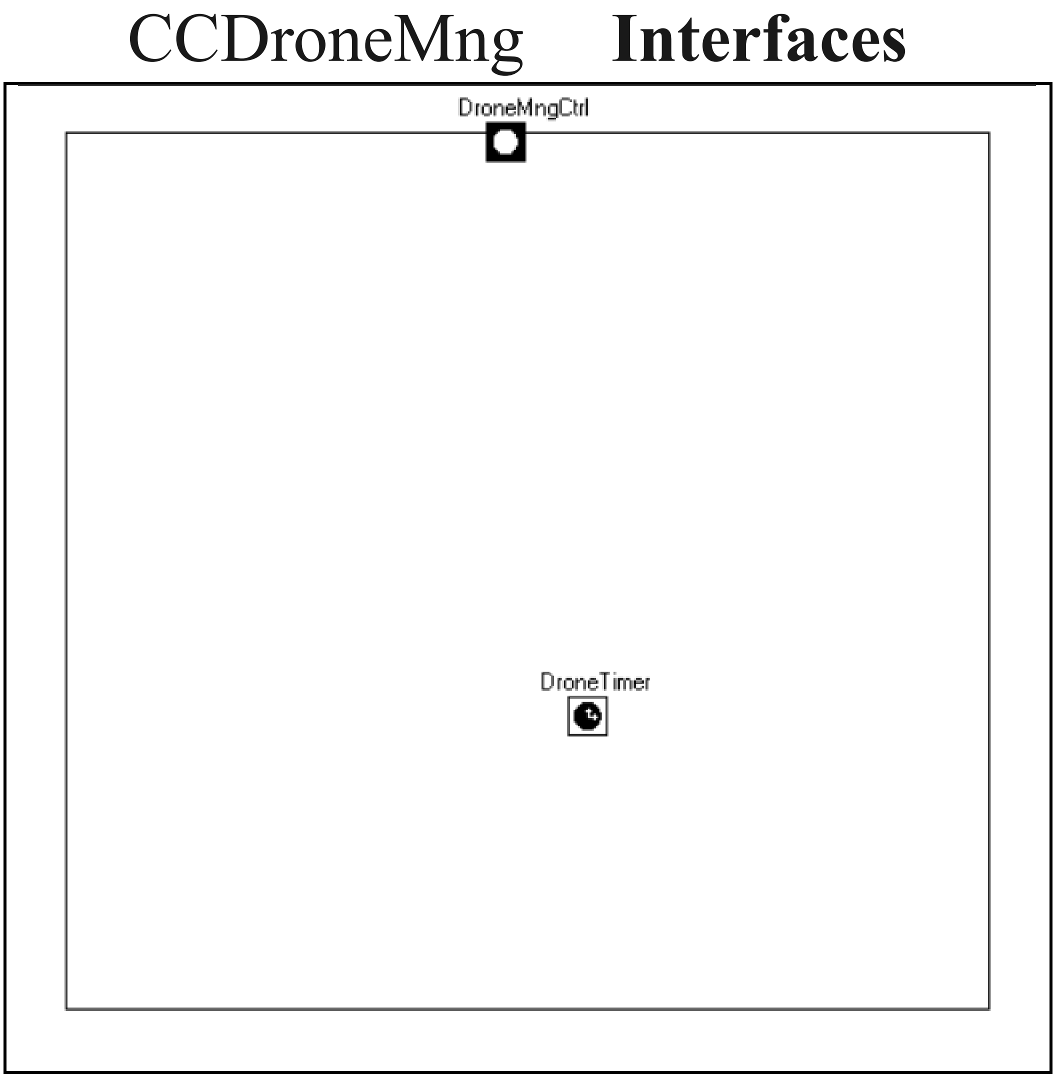
5.3 Real-Time Requirements

5.4 Software Components Design

5.4.1 CCDroneMng

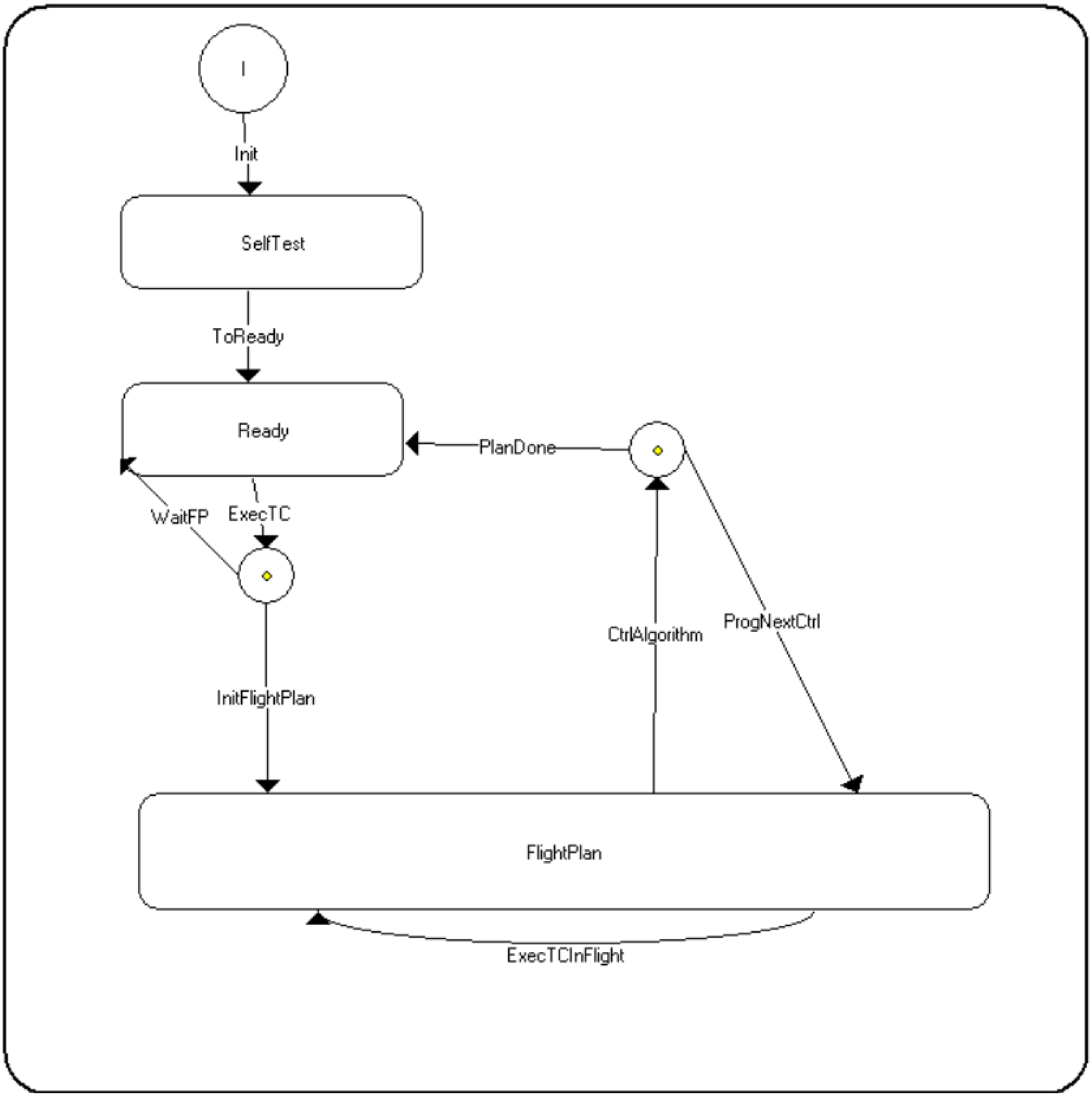
5.4.1.1 Interfaces

TODO 03 Añade los puertos que definen lasinterfaces de la clase CCDroneMng

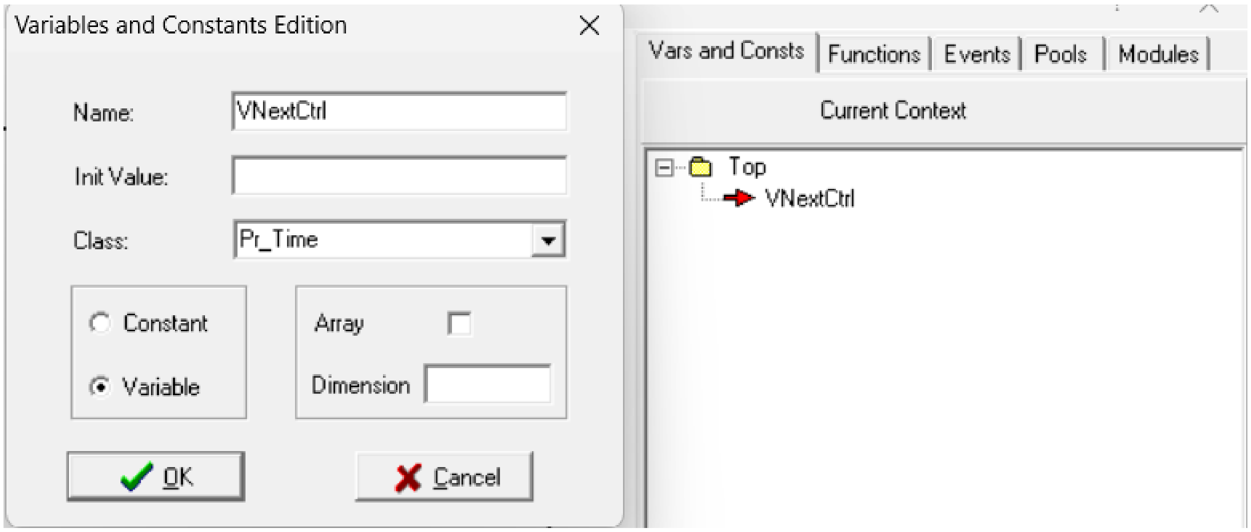


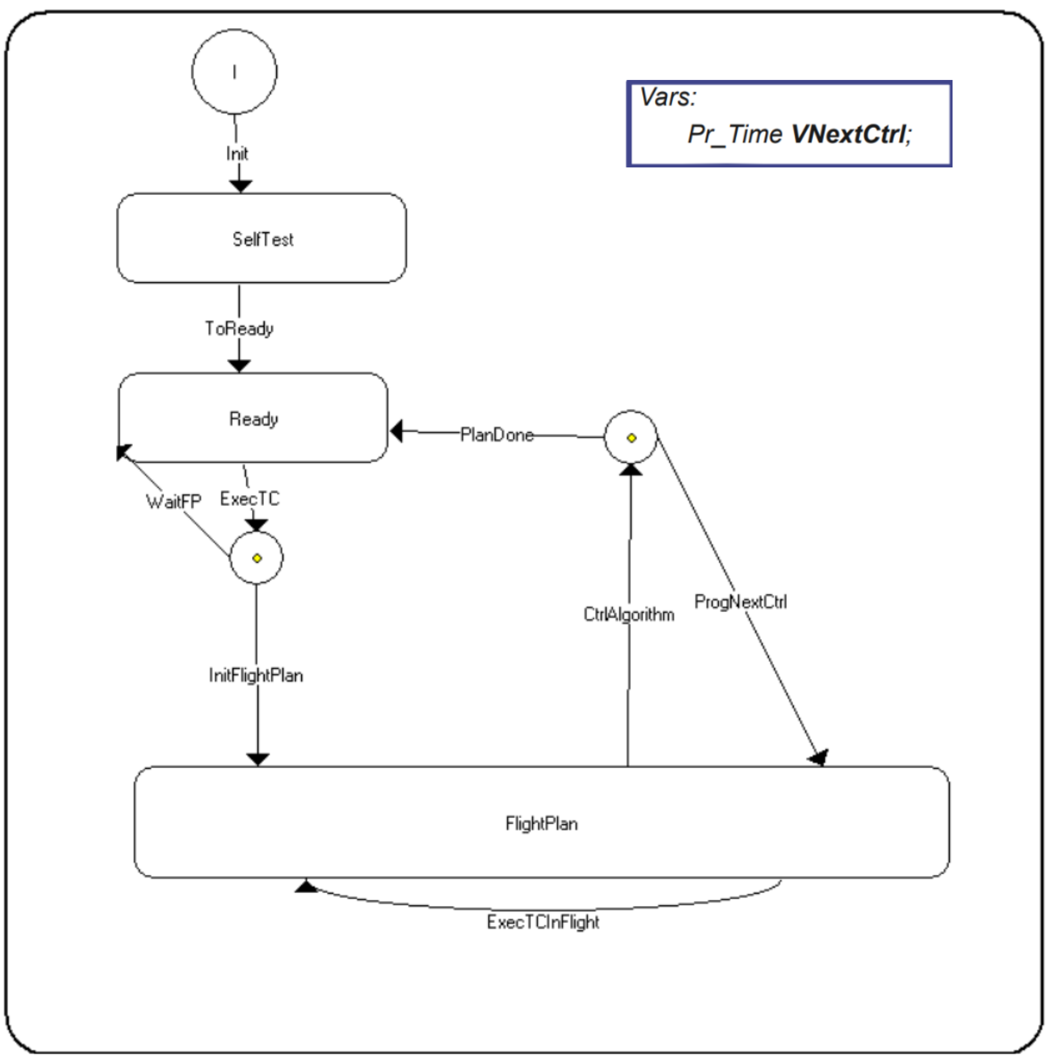
5.4.1.1 Behaviour

TODO 04 Añade el diagrama de estados de la clase CCDroneMng

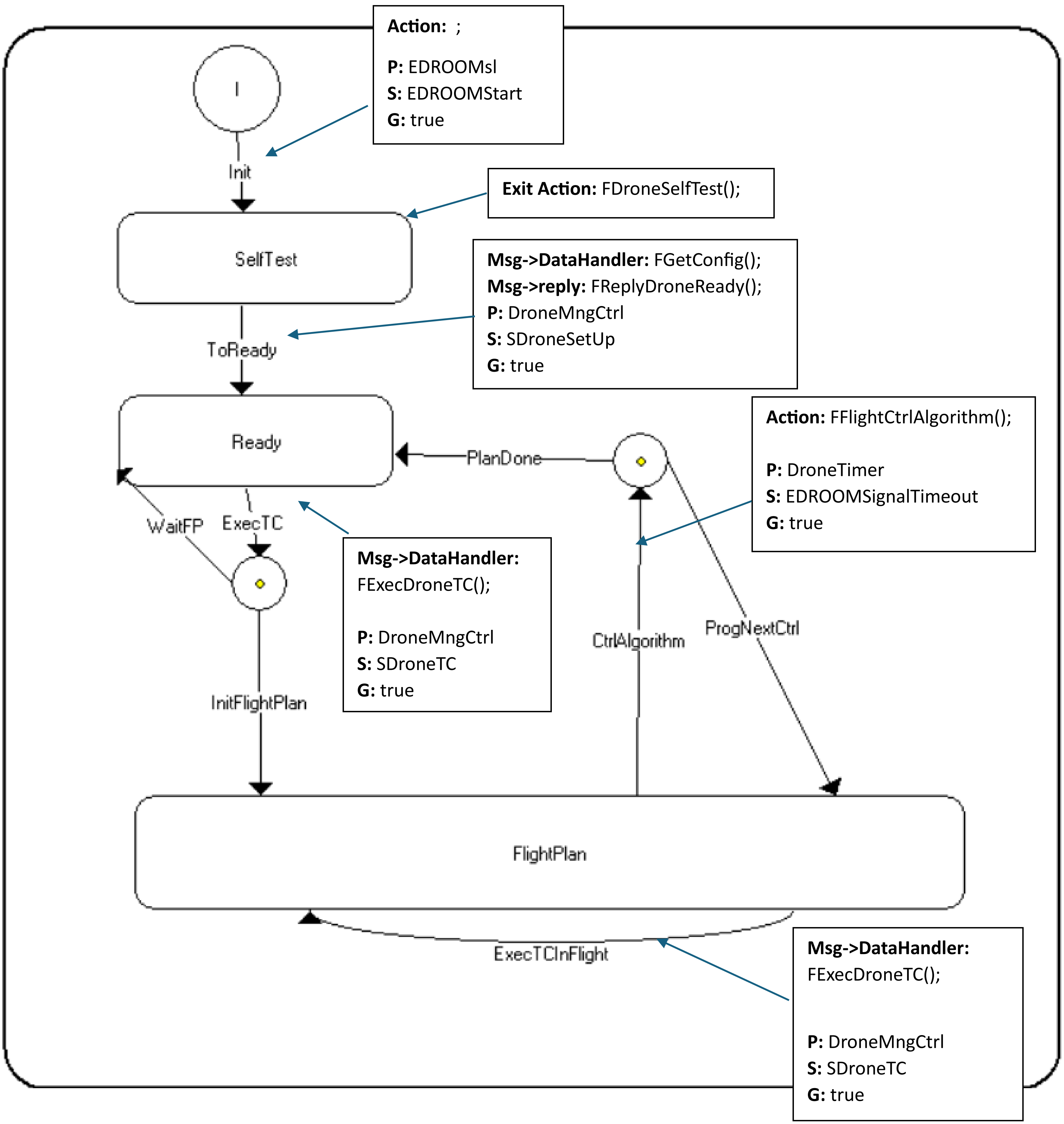


TODO 05 *Añade las variables de la clase CCDroneMng*

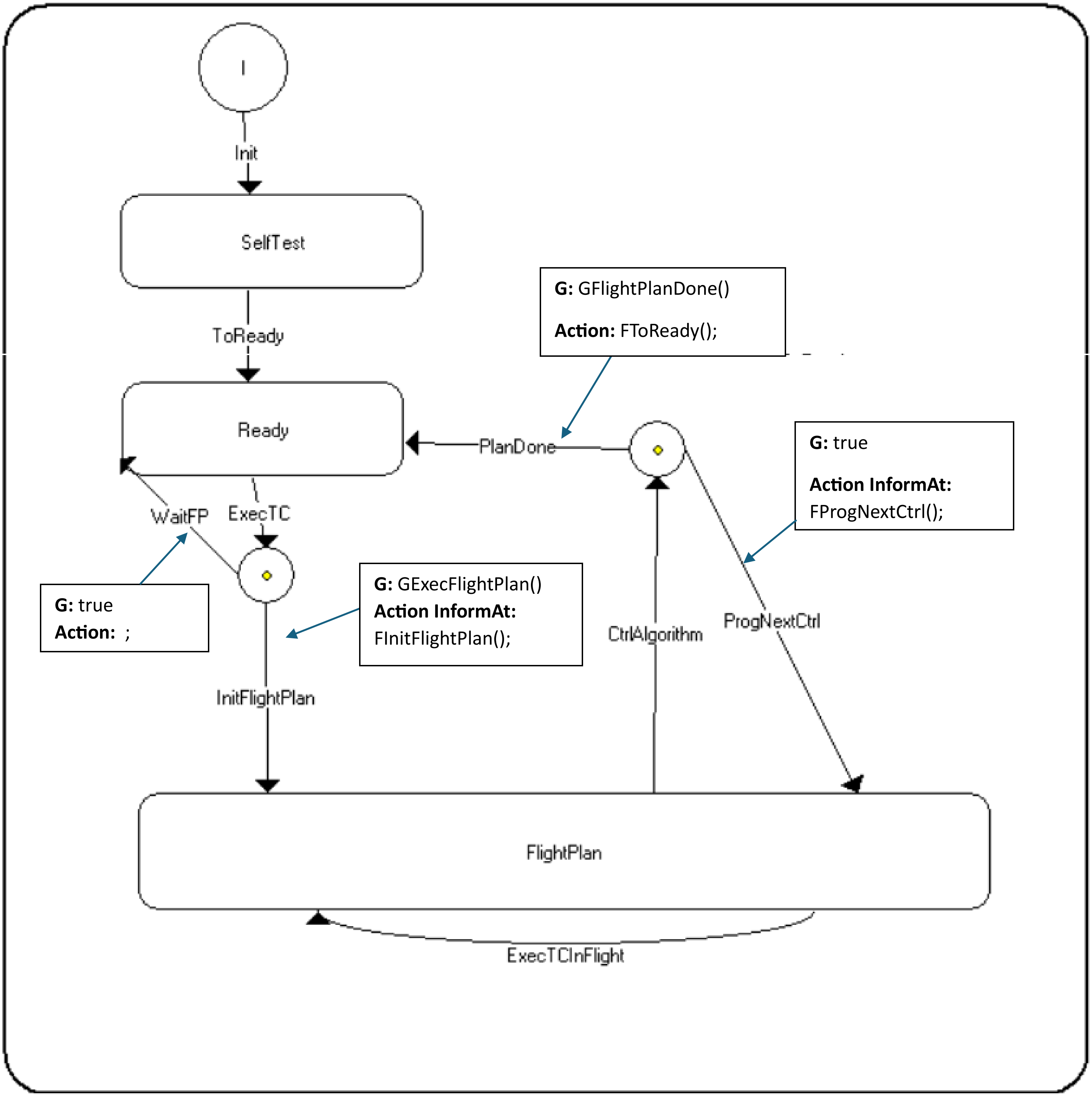




TODO 06 Añade la condición de disparo y la acción de cada una de las transiciones (utiliza el diagrama de estados varias veces si lo ves necesario)



TODO 07 Añade la guarda y la acción asociada de cada una de lasramas de un choice points (utiliza el diagrama de estados varias vecessi lo ves necesario)



TODO 08 Añade el código de las acciones y guardas (marca en azul el código que se genera automáticamente alsolicitar un servicio)

FDroneSelfTest()	void FDroneSelfTest(){
	pus_service129_drone_selftest();
	}

FExecDroneTC()	void FExecDroneTC(){
	CDTCHandler & varSDroneTC = *(CDTCHandler *)Msg->data;
	varSDroneTC.ExecDroneTC();
	}

FFlightCtrlAlgorithm()	void FFlightCtrlAlgorithm(){
	pus_service129_flight_ctrl_algorithm();
	}

FGetConfig()	void FGetConfig(){
	CDDroneConfig & varSDroneSetUp = *(CDDroneConfig *)Msg->data;
	pus_service129_setup(varSDroneSetUp);
	}

FReplyDroneReady()	void FReplyDroneReady(){
	pus_service129_drone_ready();
	
	Msg->reply(SDroneReady);
	}

FInitFlightPlan()	void FInitFlightPlan(){
	Pr_Time time;
	
	time.GetTime();
	time += Pr_Time(0, 100000);
	VNextCtrl = time;
	
	pus_service129_init_flight_plan();
	
	DroneTimer.InformAt(time);
	}

FProgNextCtrl()	void FProgNextCtrl(){
	Pr_Time time;
	
	VNextCtrl += Pr_Time(0, 100000);
	time = VNextCtrl;
	
	DroneTimer.InformAt(time);
	}

FToReady()	void FToReady(){
	pus_service129_drone_ready();
	}

GExecFlightPlan()	bool GExecFlightPlan(){
	return pus_service129_flight_plan_ready();
	}

GFlightPlanDone()	bool GFlightPlanDone(){
	return pus_service129_flight_plan_done();
	}